

ERTS: Assignment 2

Simon Møller, student number: 202204535

Mark Myrtoft Bøgelund, student number: 202206634

Date: September 29, 2026

Introduction

The purpose of this assignment is to learn the design flow for a system on a programmable chip. Using Vivado, SDK, and HLS, we explore the relationship between the Zynq Processing System (PS) and Programmable Logic (PL), interface drivers, command-line control, custom IP, and hardware acceleration.

We began by following the setup instructions to configure the ZYBO board, connect it to the development computer, and prepare the required Xilinx design tools. This was initially quite challenging, since our pc runs on Windows 11, and the old software was not fully compatible. Eventually, we got it working. This provided the hardware and software platform for the exercises that follow.

Exercise 3: Create a Console Application

Objective

In this exercise, we create a small console application for the ZYBO board. It receives commands over USB-UART and uses them to control the LEDs through the switches and timer.

Command interpreter

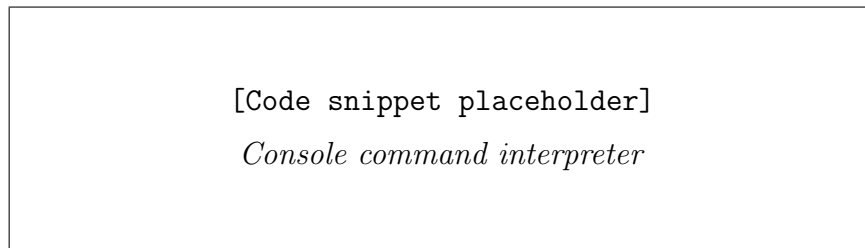


Figure 1: Console command interpreter

The application will read a command from the terminal and use a **switch** statement to select the requested operation.

Command 1: Switches to LEDs

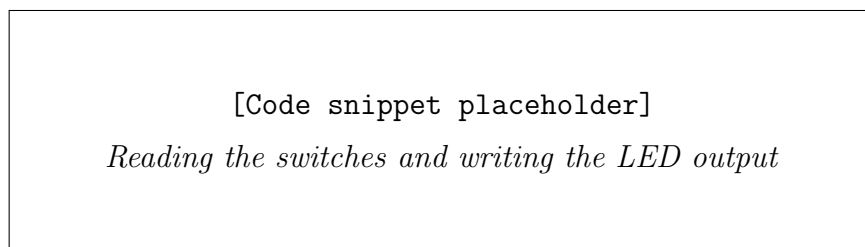


Figure 2: Reading the switches and writing the LED output

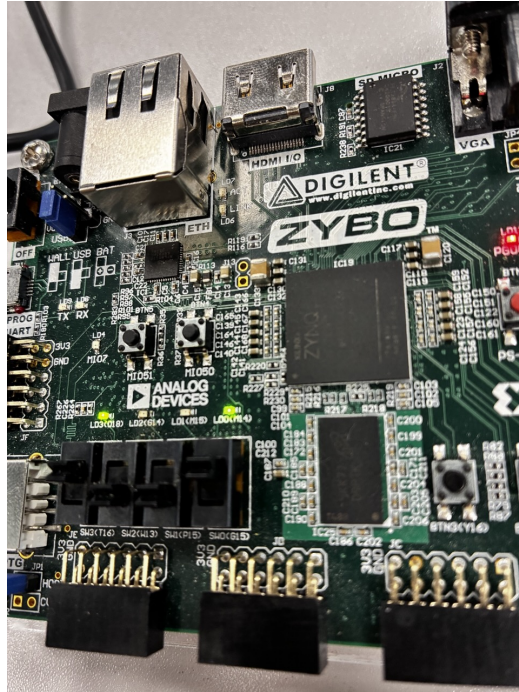


Figure 3: The LEDs reflect the binary state of the switches.

With command 1, the LEDs light up according to the switch positions.

Command 2: Timer-driven binary counter

[Code snippet placeholder]

XScuTimer initialization and one-second counter

Figure 4: XScuTimer initialization and one-second counter

Exercise 4: Create a Matrix Multiplication

Objective

In this exercise, we implement a software matrix multiplication on the ZYBO board. The program uses two 4x4 matrices, calculates the product, and reports the execution time through the USB-UART interface.

Matrix multiplication

The first matrix contains the values from 1 to 16. For the second matrix, the assignment gives B^T , so if:

$$B^T = \left(\begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 2 & 2 & 2 \\ 3 & 3 & 3 & 3 \\ 4 & 4 & 4 & 4 \end{bmatrix} \right)^T = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 \end{bmatrix}.$$

Then:

$$B = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 2 & 2 & 2 \\ 3 & 3 & 3 & 3 \\ 4 & 4 & 4 & 4 \end{bmatrix}$$

We used the matrices A and B as defined above for the multiplication, which is performed using nested loops, where each element in the result is calculated as the sum of four products.

Following the operations shown in the figure below, which we took from the assignment:

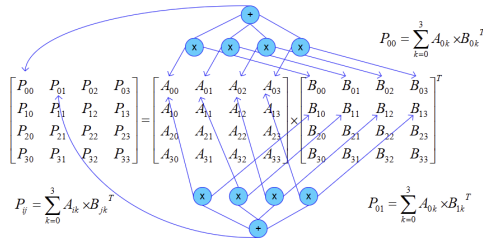


Figure 7: Matrix multiplication operations for calculating the elements of P .

We made the multiplication and transposition in the same loops. This was confusing at first, but we eventually got it and arrived at the correct result.

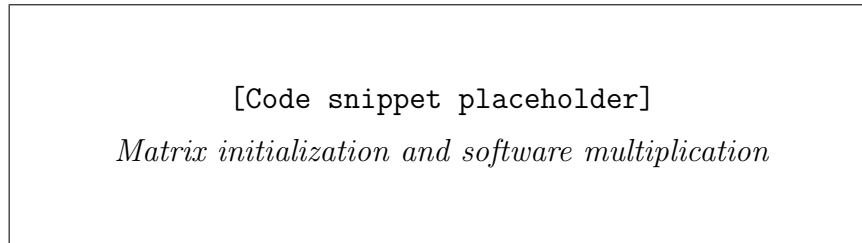


Figure 8: Matrix initialization and software multiplication

```

CMD :> -- Start of the Program --
Enter choice: 1 (SW->LEDs), 2 (Timer->LEDs), 3 (Matrix), 4 (exit)
CMD :>
Matrix A:
1      2      3      4
5      6      7      8
9      10     11     12
13     14     15     16

Matrix B:
1      1      1      1
2      2      2      2
3      3      3      3
4      4      4      4

Matrix P:
10     20     30     40
26     52     78     104
42     84     126    168
58     116    174    232

Execution time: 1240 clock ticks
CMD :>

```

Figure 9: Matrix multiplication result and execution time.

The result matrix is displayed correctly, and the software implementation takes 1102 clock ticks in this run. This result will later be used for comparison with the hardware-accelerated version.

Exercise 5: Create a Hardware IP Acceleration

Objective

In this exercise, we use a custom hardware IP to accelerate the matrix multiplication from Exercise 4. The IP performs four byte-wise multiplications and adds the results before returning a single matrix element.

Matrix IP

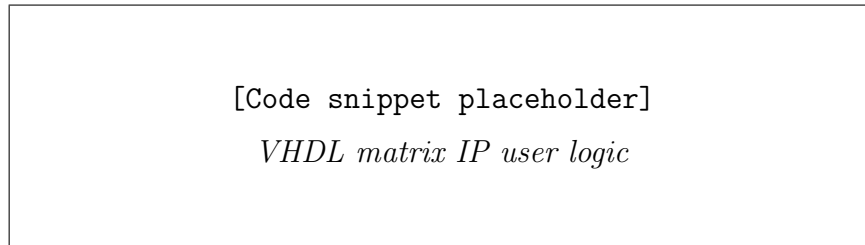


Figure 10: VHDL matrix IP user logic

The matrix IP receives two packed 32-bit values through its registers. Each value contains four matrix elements, so the IP can multiply the corresponding bytes in parallel and return their sum. The result is read through a third memory-mapped register.

Hardware matrix multiplication

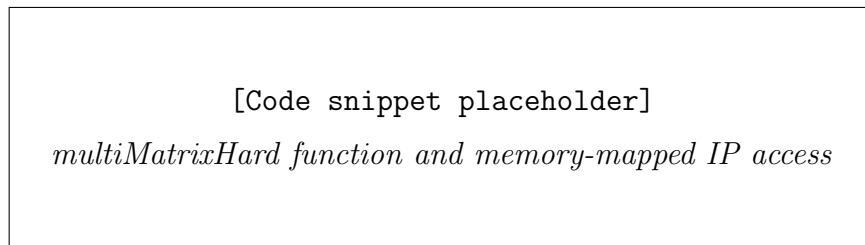


Figure 11: multiMatrixHard function and memory-mapped IP access

The hardware implementation uses the same matrix structure as the software version. Instead of calculating the four products in the processor, each row of A and each row of B are written to the IP. The returned value is then stored in the corresponding element of P .

Timing comparison

```

CMD :> -- Start of the Program --
Enter choice: 1 (SW->LEDs), 2 (Timer->LEDs), 3 (Matrix), 4 (exit)
CMD :>
Matrix A:
1      2      3      4
5      6      7      8
9      10     11     12
13     14     15     16

Matrix B:
1      1      1      1
2      2      2      2
3      3      3      3
4      4      4      4

Software result:
10     20     30     40
26     52     78     104
42     84     126    168
58     116    174    232
Software time: 1253 clock ticks

Hardware result:
10     20     30     40
26     52     78     104
42     84     126    168
58     116    174    232
Hardware time: 3981 clock ticks
CMD :> █

```

Figure 12: Comparison of the software and hardware matrix multiplication results.

Both implementations produce the same result. The software version took 1253 clock ticks, compared with 3981 ticks for the hardware version. The processing system runs at 650 MHz, while the hardware IP in the programmable logic runs at 100 MHz. The hardware time also includes transfers to and from the IP. If the hardware IP ran at 650 MHz, its ideal scaled time would be:

$$3981 \cdot \frac{100}{650} \approx 612 \text{ clock ticks.}$$

Compared with the software result, this corresponds to an ideal speedup of

$$\frac{1253}{612} \approx 2.05 \times .$$

The practical speedup would be lower because the transfer overhead remains.

Exercise 7: HLS exercise implementing a Neural Network

Objective

In this exercise, we implement a configurable two-layer neural network in C++ and use Vivado HLS to synthesize it as an IP core. The network uses fixed-point integer arithmetic, with weights scaled by 2^{15} , and applies a ReLU activation to the hidden layer.

Neural network implementation

The network has 196 inputs, 32 hidden neurons, and 10 outputs. The weights are represented in fixed-point form with 15 fractional bits. After each dot product, the result is shifted back by 15 bits; ReLU is applied to the hidden-layer values.

Listing 1: Fixed-point scaling and ReLU in the hidden layer

```
int value = static_cast<int>(sum >> BITS);  
neurons[i] = value < 0 ? 0 : value;
```

The input and weight arrays are controlled by the processor through AXI4-Lite. Switches and LEDs use direct ports, with the switch value passed through to the LEDs.

Listing 2: HLS interfaces for processor data and board I/O

```
#pragma HLS INTERFACE s_axilite port=input bundle=CTRL  
#pragma HLS INTERFACE s_axilite port=weightsI bundle=  
    CTRL  
#pragma HLS INTERFACE s_axilite port=weightsO bundle=  
    CTRL  
#pragma HLS INTERFACE s_axilite port=output bundle=CTRL  
#pragma HLS INTERFACE ap_none port=inSwitch  
#pragma HLS INTERFACE ap_none port=outLeds  
  
*outLeds = inSwitch;
```

The provided ‘neuralNet.h’ defines the function interface, and ‘testBench.cpp’ supplies test inputs and checks the C model before synthesis.

HLS results

We compared the design with and without pipelining the inner loops. In the pipelined version, an initiation interval of one is requested:

Listing 3: Pipeline directive used on the inner loops

```
#pragma HLS PIPELINE II=1
```

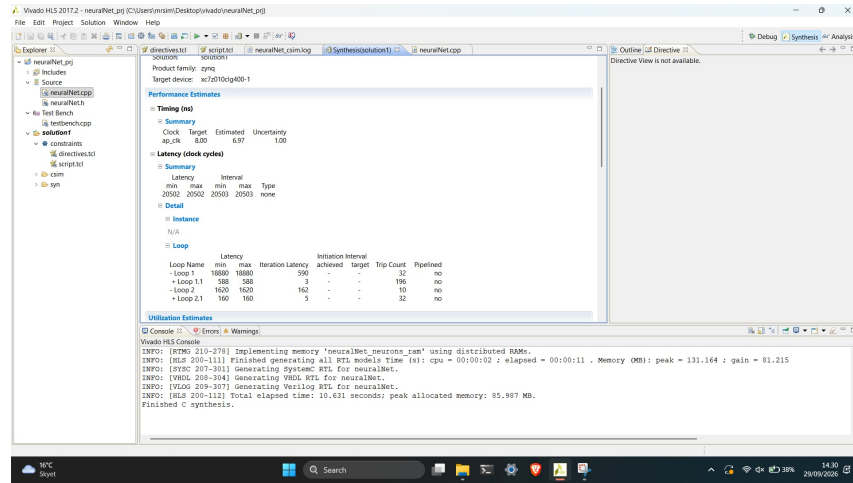


Figure 13: Vivado HLS simulation and synthesis results.

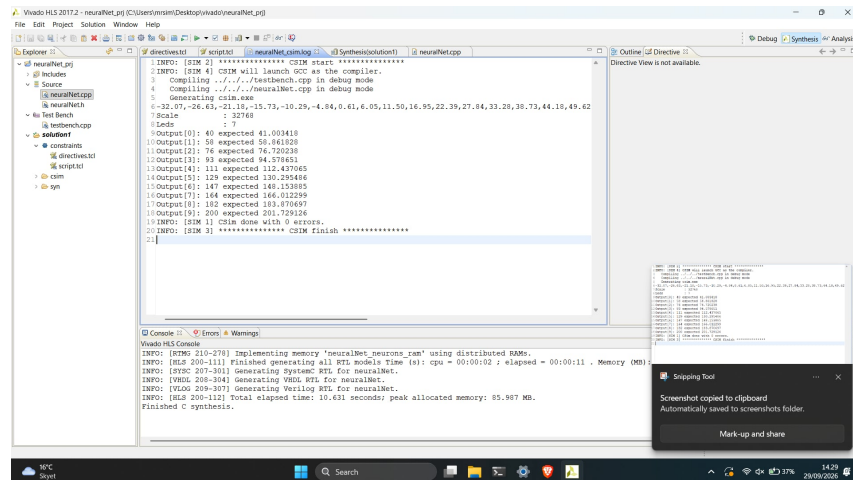


Figure 14: Simulation log for the neural network testbench.

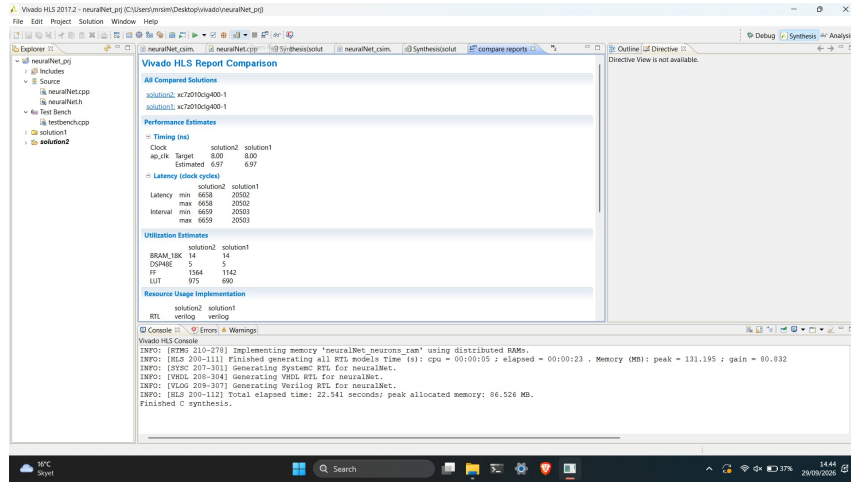


Figure 15: Comparison of the pipelined and non-pipelined HLS implementations.

ZYBO integration and timing

The synthesized IP was exported as VHDL version 1.0.1 and integrated into the Vivado ZYBO design. The board photos document the switch-to-LED pass-through. The SDK test compares software and hardware outputs and measures input, weight, compute, and output-transfer times separately, making the data-transfer cost visible.

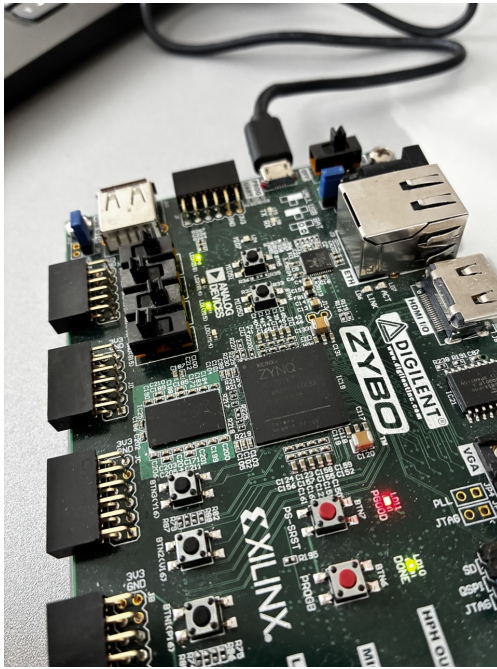


Figure 16: ZYBO switch and LED verification.

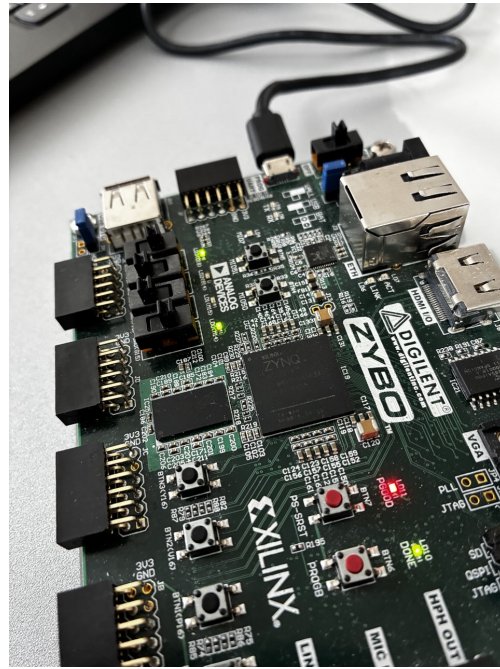


Figure 17: A second switch and LED test state.

The HLS implementations and SDK test programs are listed in the appendix. The assignment-provided header and testbench should be included there as well.

Exercise 7 source code

The following listings contain the HLS implementation without pipelining, the pipelined implementation, the direct switch-to-LED test, and the SDK software/hardware comparison program.

Listing 4: HLS neural network without pipelining

```
#include "neuralNet.h"

void neuralNet(
    char input[INPUT_LEN],
    short weightsI[WEIGHTS_I_LEN],
```

```

    short weights0[WEIGHTS_0_LEN],
    int output[OUTPUT_LEN],
    ap_int<SL_LEN> inSwitch,
    ap_int<SL_LEN> *outLeds)
{
#pragma HLS INTERFACE s_axilite port=input bundle=CTRL
#pragma HLS INTERFACE s_axilite port=weightsI bundle=
    CTRL
#pragma HLS INTERFACE s_axilite port=weights0 bundle=
    CTRL
#pragma HLS INTERFACE s_axilite port=output bundle=CTRL
#pragma HLS INTERFACE ap_none port=inSwitch
#pragma HLS INTERFACE ap_none port=outLeds
#pragma HLS INTERFACE s_axilite port=return bundle=CTRL

    int neurons[NEURONS_LEN];

    *outLeds = inSwitch;

    for (int i = 0; i < NEURONS_LEN; ++i) {
        long long sum = 0;
        const int offset = NEURONS_LEN * i;

        for (int j = 0; j < INPUT_LEN; ++j) {
            sum += static_cast<int>(input[j]) * weightsI
                [offset + j];
        }

        int value = static_cast<int>(sum >> BITS);
        neurons[i] = value < 0 ? 0 : value;
    }

    for (int i = 0; i < OUTPUT_LEN; ++i) {
        long long sum = 0;
        const int offset = OUTPUT_LEN * i;

        for (int j = 0; j < NEURONS_LEN; ++j) {
            sum += static_cast<long long>(neurons[j]) *
                weights0[offset + j];
        }
    }
}

```

```

        output[i] = static_cast<int>(sum >> BITS);
    }
}

```

Listing 5: HLS neural network with pipelining

```

#include "neuralNet.h"

void neuralNet(
    char input[INPUT_LEN],
    short weightsI[WEIGHTS_I_LEN],
    short weightsO[WEIGHTS_O_LEN],
    int output[OUTPUT_LEN],
    ap_int<SL_LEN> inSwitch,
    ap_int<SL_LEN> *outLeds)
{
    #pragma HLS INTERFACE s_axilite port=input bundle=CTRL
    #pragma HLS INTERFACE s_axilite port=weightsI bundle=
        CTRL
    #pragma HLS INTERFACE s_axilite port=weightsO bundle=
        CTRL
    #pragma HLS INTERFACE s_axilite port=output bundle=CTRL
    #pragma HLS INTERFACE ap_none port=inSwitch
    #pragma HLS INTERFACE ap_none port=outLeds
    #pragma HLS INTERFACE s_axilite port=return bundle=CTRL

    int neurons[NEURONS_LEN];

    *outLeds = inSwitch;

    hidden_neurons:
    for (int i = 0; i < NEURONS_LEN; ++i) {
        long long sum = 0;
        const int offset = NEURONS_LEN * i;

        hidden_inputs:
        for (int j = 0; j < INPUT_LEN; ++j) {
            #pragma HLS PIPELINE II=1
            sum += static_cast<int>(input[j]) * weightsI
                [offset + j];
        }
    }
}

```

```

    }

    int value = static_cast<int>(sum >> BITS);
    neurons[i] = value < 0 ? 0 : value;
}

output_neurons:
for (int i = 0; i < OUTPUT_LEN; ++i) {
    long long sum = 0;
    const int offset = OUTPUT_LEN * i;

    output_inputs:
    for (int j = 0; j < NEURONS_LEN; ++j) {
#pragma HLS PIPELINE II=1
        sum += static_cast<long long>(neurons[j]) *
            weights0[offset + j];
    }

    output[i] = static_cast<int>(sum >> BITS);
}
}

```

Listing 6: ZYBO switch-to-LED verification

```

#include "xil_printf.h"

int main(void)
{
    xil_printf("\r\n");
    xil_printf("NeuralNet_FPGA_test\r\n");
    while (1)
    {
        /*
         * The NeuralNet is connected directly to the
         * switches and LEDs in the FPGA fabric.
         *
         * Nothing needs to be done by the processor.
         */
    }

    return 0;
}

```

```
}
```

Listing 7: SDK software and hardware timing comparison

```
/*
*****

*
* neuralNet.c
*
* Measures and compares:
*
*   1. Software neural network execution on ARM Cortex-
*      A9
*   2. Hardware IP:
*      - input transfer time
*      - weight transfer time
*      - neural network computation time
*      - output transfer time
*      - total hardware execution time
*
* The measurements can be used to identify the
*   bottleneck in the
* hardware-accelerated implementation.
*
*****
*/

#include <stdio.h>
#include <stdlib.h>

#include "xil_printf.h"
#include "XScuTimer.h"
#include "xneuralnet.h"

/*
=====
*/
/* Configuration
```

```

    */
/*
=====
    */

#define ONE_SECOND 333000000

#define BITS          15

#define INPUT_LEN      196
#define NEURONS_LEN    32
#define OUTPUT_LEN     10

#define WEIGHTS_I_LEN  (INPUT_LEN * NEURONS_LEN)
#define WEIGHTS_O_LEN  (NEURONS_LEN * OUTPUT_LEN)

/*
=====
    */
/* Hardware IP instance

    */
/*
=====
    */

XNeuralnet neuralNetHLS;

/*
=====
    */
/* Timer

    */
/*
=====
    */

```

```

XScuTimer Timer;

XScuTimer_Config *ConfigPtr;
XScuTimer *TimerInstancePtr = &Timer;

/*
 * Timer is a down-counter.
 * startTimer() stores the starting counter value.
 * stopTimer() returns the number of timer ticks elapsed
 */

unsigned int startTimer(void)
{
    unsigned int start;

    start = XScuTimer_GetCounterValue(TimerInstancePtr);
    return start;
}

unsigned int stopTimer(unsigned int start)
{
    unsigned int stop;

    stop = XScuTimer_GetCounterValue(TimerInstancePtr);

    /*
     * Timer counts down.
     */
    return start - stop;
}

void initTimer(void)
{
    int Status;

    ConfigPtr = XScuTimer_LookupConfig(

```

```

        XPAR_PS7_SCUTIMER_0_DEVICE_ID
    );

    Status = XScuTimer_CfgInitialize(
        TimerInstancePtr,
        ConfigPtr,
        ConfigPtr->BaseAddr
    );

    if (Status != XST_SUCCESS) {
        print("Timer initialization failed\r\n");
        return;
    }

    XScuTimer_LoadTimer(TimerInstancePtr, ONE_SECOND);

    XScuTimer_EnableAutoReload(TimerInstancePtr);

    XScuTimer_Start(TimerInstancePtr);
}

/*
=====
*/
/* Neural network data
*/
/*
=====
*/

signed char inputs[INPUT_LEN];

short weightsIS[WEIGHTS_I_LEN];
float weightsI[WEIGHTS_I_LEN];

short weightsOS[WEIGHTS_O_LEN];
float weightsO[WEIGHTS_O_LEN];

```

```
int outputsS[OUTPUT_LEN];
float outputsF[OUTPUT_LEN];
```

```
/*
```

```
=====
```

```
*/
```

```
/* Software implementation
```

```
*/
```

```
/*
```

```
=====
```

```
*/
```

```
void neuralNetFloat(
    signed char input[INPUT_LEN],
    float weightsI[WEIGHTS_I_LEN],
    float weightsO[WEIGHTS_O_LEN],
    float output[OUTPUT_LEN]
```

```
)
```

```
{
```

```
    float neurons[NEURONS_LEN];
```

```
    float neuron;
```

```
    float result;
```

```
    int i;
```

```
    int j;
```

```
    int offsetI;
```

```
    int offsetO;
```

```
/*
```

```
-----
```

```
*/
```

```
/* Input layer
```

```
*/
```

```
/*
```

```
-----
```

```

        */

for (i = 0; i < NEURONS_LEN; i++) {

    neuron = 0;

    offsetI = NEURONS_LEN * i;

    for (j = 0; j < INPUT_LEN; j++) {

        neuron +=
            input[j] *
            weightsI[j + offsetI];
    }

    /* ReLU */
    if (neuron < 0)
        neuron = 0;

    neurons[i] = neuron;
}

/*
-----
    */
/* Output layer

    */
/*
-----
    */

for (i = 0; i < OUTPUT_LEN; i++) {

    result = 0;

    offsetO = OUTPUT_LEN * i;

    for (j = 0; j < NEURONS_LEN; j++) {

```

```

        result +=
            neurons[j] *
            weights0[j + offset0];
    }

    output[i] = result;
}

/*
=====
    */
/* Hardware implementation                                     */
/*
=====
    */

/*
=====
    */

/*
 * The hardware implementation is deliberately split
 * into individual
 * measurements.
 *
 * This allows us to determine whether the bottleneck is
 * :
 *
 * - transferring inputs
 * - transferring weights
 * - executing the neural network
 * - reading the output
 *
 */

void neuralNetHW(
    signed char input[INPUT_LEN],
    short wI[WEIGHTS_I_LEN],
    short wO[WEIGHTS_O_LEN],
    int output[OUTPUT_LEN]

```

```

)
{
    unsigned int start;

    unsigned int inputTransferTicks;
    unsigned int weightTransferTicks;
    unsigned int outputTransferTicks;
    unsigned int computeTicks;

    unsigned int totalTicks;

    /* Wait until IP is ready */

    while (XNeuralnet_IsReady(&neuralNetHLS) == 0);

    /*
        =====
        */
    /* Transfer input data

        */
    /*
        =====
        */

    start = XScuTimer_GetCounterValue(TimerInstancePtr);

    XNeuralnet_Write_input_r_Words(
        &neuralNetHLS,
        0,
        (int *)input,
        INPUT_LEN / 4
    );

    inputTransferTicks = stopTimer(start);

    /*

```

```

=====
    */
    /* Transfer input-layer weights
                                                    */
    /*
=====
    */

start = XScuTimer_GetCounterValue(TimerInstancePtr);

XNeuralnet_Write_weightsI_Words(
    &neuralNetHLS,
    0,
    (int *)wI,
    WEIGHTS_I_LEN / 2
);

weightTransferTicks = stopTimer(start);

/*
=====
    */
    /* Transfer output-layer weights
                                                    */
    /*
=====
    */

start = XScuTimer_GetCounterValue(TimerInstancePtr);

XNeuralnet_Write_weightsO_Words(
    &neuralNetHLS,
    0,
    (int *)wO,
    WEIGHTS_O_LEN / 2
);

weightTransferTicks += stopTimer(start);

```

```

/*
=====
*/
/* Execute neural network hardware IP */
/*
=====
*/

start = XScuTimer_GetCounterValue(TimerInstancePtr);

XNeuralnet_Start(&neuralNetHLS);

while (XNeuralnet_IsDone(&neuralNetHLS) == 0);

computeTicks = stopTimer(start);

/*
=====
*/
/* Read output from hardware */
/*
=====
*/

start = XScuTimer_GetCounterValue(TimerInstancePtr);

XNeuralnet_Read_output_r_Words(
    &neuralNetHLS,
    0,
    (int *)output,
    OUTPUT_LEN
);

outputTransferTicks = stopTimer(start);

```

```

/*
=====
*/
/* Total hardware time

*/
/*
=====
*/

totalTicks =
    inputTransferTicks +
    weightTransferTicks +
    computeTicks +
    outputTransferTicks;

/*
=====
*/
/* Print measurements

*/
/*
=====
*/

printf("\r\n");
printf("-----_Hardware_Neural_Network_Timing_-----\r\n");

printf("Input_transfer_00000000:_%u_ticks\r\n",
    inputTransferTicks);

printf("Weight_transfer_00000000:_%u_ticks\r\n",
    weightTransferTicks);

printf("HW_computation_00000000:_%u_ticks\r\n",
    computeTicks);

```

```

        printf("Output_transfer: %u ticks\r\n",
               outputTransferTicks);

        printf("HW_total: %u ticks\r\n",
               totalTicks);

        printf("-----\r\n");
    }

    /*
    =====
    */
    /* Test neural network

    */
    /*
    =====
    */

void testNeuralNet(void)
{
    unsigned int start;
    unsigned int softwareTicks;

    int scale;
    int i;

    float diff;

    scale = (1 << BITS);

    /*
    =====
    */
    /* Generate test input data
    */

```

```

/*
=====
*/

for (i = 0; i < INPUT_LEN; i++) {

    inputs[i] = i;
}

/*
=====
*/
/* Generate input-layer weights
                                                    */
/*
=====
*/

for (i = 0; i < WEIGHTS_I_LEN; i++) {

    weightsI[i] = 0.0001f * i;

    /*
     * Convert floating-point weights to fixed point
     */
    weightsIS[i] =
        (short)(weightsI[i] * scale + 0.5f);
}

/*
=====
*/
/* Generate output-layer weights
                                                    */
/*
=====
*/

```

```

for (i = 0; i < WEIGHTS_0_LEN; i++) {

    weights0[i] = 0.001f * i;

    /*
     * Convert floating-point weights to fixed point
     */
    weightsOS[i] =
        (short)(weights0[i] * scale + 0.5f);
}

/*
=====
*/
/* SOFTWARE IMPLEMENTATION
*/
/*
=====
*/

printf("\r\n");
printf("=====\r\n");
printf("Software□implementation\r\n");
printf("=====\r\n");

start = XScuTimer_GetCounterValue(TimerInstancePtr);

neuralNetFloat(
    inputs,
    weightsI,
    weights0,
    outputsF
);

softwareTicks = stopTimer(start);

```

```

printf("CPU_computation: %u ticks\r\n",
       softwareTicks);

/*
=====
*/
/* HARDWARE IMPLEMENTATION
*/
/*
=====
*/

printf("\r\n");
printf("=====\r\n");
printf("Hardware implementation\r\n");
printf("=====\r\n");

neuralNetHW(
    inputs,
    weightsIS,
    weightsOS,
    outputsS
);

/*
=====
*/
/* Compare results
*/
/*
=====
*/

printf("\r\n");

```

```

printf("=====\r\n");
printf("Result comparison\r\n");
printf("=====\r\n");

printf("Fixed-point scale: %d\r\n\r\n", scale);

diff = 0;

for (i = 0; i < OUTPUT_LEN; i++) {

    printf(
        "Output [%d] = %d Software = %.2f\r\n",
        i,
        outputsS[i],
        outputsF[i]
    );

    diff += abs(
        outputsS[i] -
        (int)outputsF[i]
    );
}

if (diff < 10)
    printf("\r\nResult: CORRECT\r\n");
else
    printf("\r\nResult: DIFFERENT\r\n");

/*
=====
*/
/* Speedup
*/
/*
=====

```

```

        */

/*
 * Note:
 *
 * To calculate hardware total time we need the
 *   individual HW
 * measurements again. For this reason the timing
 *   values can also
 * be returned from neuralNetHW if numerical speedup
 *   is required.
 *
 * The important comparison is:
 *
 *   CPU computation
 *
 * versus
 *
 *   HW computation
 *
 * and separately:
 *
 *   CPU computation
 *
 * versus
 *
 *   HW transfer + computation
 */
}

/*
=====
        */
/* Main

        */
/*
=====
        */

```

```

int main(void)
{
    int depth;
    int bitwidth;

    print("=====\r\n");
    print("Neural_Network_Software/Hardware_Test\r\n");
    print("=====\r\n");

    /* Initialize timer */

    initTimer();

    /*
=====
    */
    /* Initialize neural network HLS IP
                                                    */

    /*
=====
    */

    if (XNeuralnet_Initialize(
        &neuralNetHLS,
        XPAR_NEURALNET_0_DEVICE_ID
    ) != XST_SUCCESS)
    {
        print("Neural_network_initialization_failed\r\n");
        return XST_FAILURE;
    }

    /*

```

```

=====
    */
    /* Display IP interface information
                                           */
    /*
=====
    */

    depth =
        XNeuralnet_Get_input_r_Depth(
            &neuralNetHLS
        );

    bitwidth =
        XNeuralnet_Get_input_r_BitWidth(
            &neuralNetHLS
        );

    printf(
        "Input: depth=%d, bitwidth=%d\r\n",
        depth,
        bitwidth
    );

    depth =
        XNeuralnet_Get_weightsI_Depth(
            &neuralNetHLS
        );

    bitwidth =
        XNeuralnet_Get_weightsI_BitWidth(
            &neuralNetHLS
        );

    printf(
        "Input weights: depth=%d, bitwidth=%d\r\n",
        depth,
        bitwidth
    );

```

```

depth =
    XNeuralnet_Get_weights0_Depth(
        &neuralNetHLS
    );

bitwidth =
    XNeuralnet_Get_weights0_BitWidth(
        &neuralNetHLS
    );

printf(
    "Output_weights: depth=%d, bitwidth=%d\r\n",
    depth,
    bitwidth
);

/*
=====
*/
/* Run test

*/
/*
=====
*/

testNeuralNet();

return 0;
}

```